CASE STUDY · WHY PRODUCTION IS SLOW

The report that got slower every month

The same product, eighteen months on. Two hundred thousand users, two million tasks.

WHAT HAPPENED

Every Monday morning an internal report goes out to the customer success team: every user, how many tasks they are carrying, when they last signed up. It is used to spot accounts that have gone quiet.

At launch it took under a second. Nobody thought about it again.

It now takes so long that the scheduled job is killed before it finishes. Customer success has not had the report in three weeks and has started asking for it by hand.

Two engineers have already looked at it. One **added an index on the created_at column**; it made no measurable difference. The other **added four more indexes** and reported that writes across the product got noticeably slower, so they were rolled back.

Nothing about the report has changed since it was written. Only the amount of data has.

WHAT YOU HAVE BEEN TOLD

- The report is one page of SQL — six queries, run in sequence.
- The tasks table has grown from about 40,000 rows to **2,000,000**.
- **An index on created_at already exists** and the report still does not use it.
- One of the six queries is for a search box the team added last year. It has always been slow and everyone has quietly accepted it.
- Adding indexes was tried and made things worse elsewhere.
- You have the schema, the row counts, and all six queries.

YOUR FIRST HOUR

1 **Why did adding an index on created_at change nothing?**

2 **Adding four indexes made writes slower. Was that the wrong instinct, or the wrong indexes?**

3 **How would you decide which queries to fix first?**

4 **One of the six cannot be fixed by rewriting it. What do you tell the product manager?**

5 **What would you measure before changing anything?**

Composite of real incidents. Every row count and query shape here appears in the worksheet that follows, running against a real database.

discuss first · then the worksheet · then run it

Why Production Is Slow.

Name _____ Date _____

Six queries below are real shapes that ship to production every day. For each one, **estimate how many rows the database has to look at**, then write a version that does less work and say which index it needs. **Estimate before you run anything** — one of the six cannot be fixed by rewriting it at all.

THE DATABASE

users	200,000
id · email · name · country · created_at	
workspaces	5,000
id · name	
projects	40,000
id · workspace_id · name · created_at	
collaborators	280,320
id · project_id · user_id	
tasks	2,000,000
id · project_id · assignee_id · estimate_hours · status · created_at	

INDEXES THAT ALREADY EXIST

every primary key
`tasks(created_at)` `users(email)`
`tasks(project_id)` `collaborators(project_id)`
not indexed: `tasks.assignee_id`, `projects.workspace_id`, `tasks.estimate_hours`

HOW TO ESTIMATE

read every row of a table	that many rows
find by index, equality	about 1, plus the rows that match
join, no index on the key	rows(A) × rows(B)
join, index on the key	rows(A) × matches each
subquery in SELECT or WHERE	it runs **once per outer row**
sort with no index	every row, then sorted

RUN IT AND CHECK YOURSELF

```
git clone https://github.com/mchoi-altitude/\
  training_slow-queries-lab
cd training_slow-queries-lab
docker compose up -d --build
docker compose exec db psql -U lab -d collab
collab=# EXPLAIN <your query>;
```

Needs **Docker Desktop**. First start seeds 2M tasks, about a minute. **EXPLAIN** prints what the planner thinks each step will cost — compare it with the number you wrote down.

1 Count each user's tasks

```
SELECT u.name,
  (SELECT count(*) FROM tasks t
   WHERE t.assignee_id = u.id) AS n
FROM users u;
```

rows examined _____
a better query

now _____ index to add _____

2 Tasks created on one day

```
SELECT count(*) FROM tasks
WHERE DATE(created_at) = '2026-03-15';
```

rows examined _____
a better query

now _____ index to add _____

3 Users on gmail

```
SELECT * FROM users
WHERE email LIKE '%@gmail.com';
```

rows examined _____
a better query

now _____ index to add _____

4 The ten longest tasks

```
SELECT * FROM tasks
ORDER BY estimate_hours DESC
LIMIT 10;
```

rows examined _____
a better query

now _____ index to add _____

5 Users never assigned a task

```
SELECT * FROM users
WHERE id NOT IN
  (SELECT assignee_id FROM tasks);
```

rows examined _____
a better query

now _____ index to add _____

6 Every task in one workspace

```
SELECT count(*)
FROM tasks t, projects p
WHERE p.id = t.project_id
  AND p.workspace_id = 1;
```

rows examined _____
a better query

now _____ index to add _____

CASE STUDY · DEATH BY A THOUSAND QUERIES

The side nav that took nine seconds

A project collaboration product, about two years old. Roughly four thousand paying workspaces.

WHAT HAPPENED

The left-hand navigation lists your projects. Each row shows the project name, how many collaborators it has, and how many tasks are in it. It is the first thing anybody sees after logging in.

It was written in an afternoon, by a developer working against his own workspace: **three projects, five tasks between them**. It felt instant. Code review passed. It shipped, and for eighteen months nobody said anything about it.

Last Tuesday your largest customer — an agency, forty active projects, a couple of hundred tasks in each — opened a ticket saying the application **“hangs on load.”** They have been living with it for a while, they say. They assumed it was their network.

The engineer who wrote the side nav left the company in March.

WHAT YOU HAVE BEEN TOLD

- The customer says the page takes **roughly nine seconds**. They are not exaggerating; they sent a screen recording.
- **Support cannot reproduce it.** On their test account the page loads immediately.
- The database team has looked. They report **no slow queries** — nothing in the slow query log, nothing above 50 ms.
- Database CPU is at 30%. Memory is fine. No alerts fired.
- Nothing was deployed last week. Nobody changed anything.
- The account is up for renewal in five weeks.

YOUR FIRST HOUR

1 **Where do you look first, and why there?**

2 **The database team says there are no slow queries. Are they wrong?**

3 **Support cannot reproduce it. What is different about their account?**

4 **What would you ask the customer to send you?**

5 **You are on the call in one hour. What do you say?**

Based on a real incident. The company and the people are invented; the bug, the way it was found and the numbers in the worksheet that follows are not.

discuss first · then the worksheet · then run it

Death by a Thousand Queries.

Name _____ Date _____

Both of these shipped to production. **Every single query in them is fast** — indexed, sub-millisecond, nothing a query plan would complain about. Count the **queries**, not the rows; one round trip costs about **0.4 ms**. **Write your prediction down before you run anything.** Being wrong on paper is free.

1 The project side nav

PROJECTS		
Website Redesign	8	120
Mobile App	8	120
Q3 Marketing	8	120
… 37 more projects		

Workspace 1: **40 projects**, each with **8 collaborators** and **120 tasks**. Those two little numbers on every row are the whole problem.

```
projects = query("SELECT * FROM projects WHERE workspace_id=?")
for p in projects:
    collabs = query("SELECT * FROM collaborators WHERE project_id=?", p.id)
    for c in collabs:
        user = query("SELECT * FROM users WHERE id=?", c.user_id)
    tasks = query("SELECT * FROM tasks WHERE project_id=?", p.id)
    for t in tasks:
        who = query("SELECT * FROM users WHERE id=?", t.assignee_id)
```

queries = 1 + _____ + _____ + _____ + _____
total _____ at 0.4 ms each → _____ s

the one query that replaces all of it

It was tested with three projects and five tasks. It felt instant.

NOW RUN IT — PREDICT FIRST

```
curl -s localhost:5002/api/sidenav/slow | jq '{queries, ms}'
curl -s localhost:5002/api/sidenav/fast | jq '{queries, ms}'
```

you predicted _____ it actually did _____

2 The top three projects

MOST ACTIVE THIS WEEK	
1 API Migration	412
2 Website Redesign	388
3 Mobile App	355

Three rows on screen. The same **40 projects** behind them.

```
projects = query("SELECT id,name FROM projects WHERE workspace_id=?")
for p in projects:
    p.tasks = query("SELECT count(*) FROM tasks WHERE project_id=?", p.id)

for i in range(len(projects)):
    for j in range(len(projects)-1):
        a = query("SELECT count(*) FROM tasks WHERE project_id=?", projects[j].id)
        b = query("SELECT count(*) FROM tasks WHERE project_id=?", projects[j+1].id)
        if a < b: swap(projects, j, j+1)
show(projects[:3])
```

queries = 1 + _____ + _____
total _____ at 0.4 ms each → _____

the one query that replaces all of it

The task counts were **already fetched** in the first loop, then thrown away. **The database has a sort of its own, and an index. What would it have cost?**

NOW RUN IT — PREDICT FIRST

```
curl -s localhost:5002/api/top-projects/slow | jq '{queries, ms}'
curl -s localhost:5002/api/top-projects/fast | jq '{queries, ms}'
```

you predicted _____ it actually did _____

SCENARIO 2, AS THE WORKSPACE GROWS

PROJECTS	QUERIES	TIME
5		
10		
20		
40		

WHAT TO LOOK FOR IN YOUR OWN CODE

- ☐ a query inside a **for** loop
- ☐ a query inside a **sort comparator**
- ☐ fetching whole rows only to **count** them
- ☐ asking again for something you **already have**
- ☐ code that was only ever tested with **three** of something

Nothing here is a slow query. Every one of them is a fast query, asked far too many times. This is the bug a query plan **cannot show you** — and it is the one juniors ship most.

Setting it up

Needs **Docker Desktop** running. Nothing else to install — the database and the app both come up in containers.

1 Get it

```
git clone https://github.com/mchoi-altitude/\
  training_slow-queries-lab
cd training_slow-queries-lab
```

2 Start it

```
docker compose up -d --build
```

First start seeds **200,000 users**, **40,000 projects** and **2,000,000 tasks**. About a minute. Later starts are instant.

3 Check it is alive

```
curl -s localhost:5002/api/stats
```

App on **:5002**, Postgres on **:5436** (`lab/lab`). Workspace **1** is the one both endpoints use.

? If something is wrong

Port already in use — change it in `docker-compose.yml`.
Start over — `docker compose down -v`, then up again.
Done for the day — `docker compose down`.